Assignment-1

NAME: V.Nageshwari

ROLL NO: 2303A52063

COURSE: AI Assisted Coding

Question 1: Zero-Shot Prompting (Palindrome Number Program) Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.
Task:

• Record the AI-generated code.

• Test the code with multiple inputs.

• Identify any logical errors or missing edge-case handling.

PROMPT

```python
#take input from user as number  , and check whether the number is palndrome or not
def is_palindrome():
    num = int(input("Enter a number: "))
    original_num = num
    reversed_num = 0

    while num > 0:
        digit = num % 10
        reversed_num = reversed_num * 10 + digit
        num //= 10

    if original_num == reversed_num:
        print(f"{original_num} is a palindrome.")
    else:
        print(f"{original_num} is not a palindrome.")
is_palindrome()
```

OUTPUT

```
PS C:\Users\harsh\OneDrive\Documents\AIAC>  & 'c:\office\miniforge\python.exe' 'c:\Users\harsh\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60573' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 121
121 is a palindrome.
PS C:\Users\harsh\OneDrive\Documents\AIAC>
```

ANALYSISI

- The zero-shot prompt gave a correct program for checking palindrome numbers.
- The program works well for normal positive numbers.
- It does not clearly handle negative numbers or wrong input types.
- This shows that the code works, but some special cases are missing.

Question 2: One-Shot Prompting (Factorial Calculation) Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

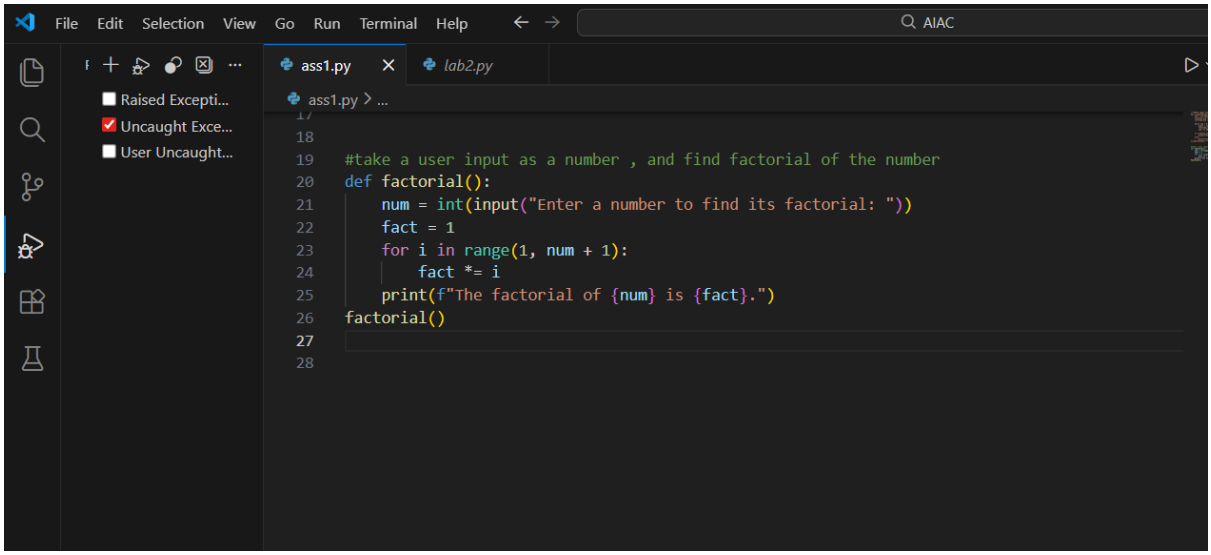
Input: 5 → Output: 120

Task:

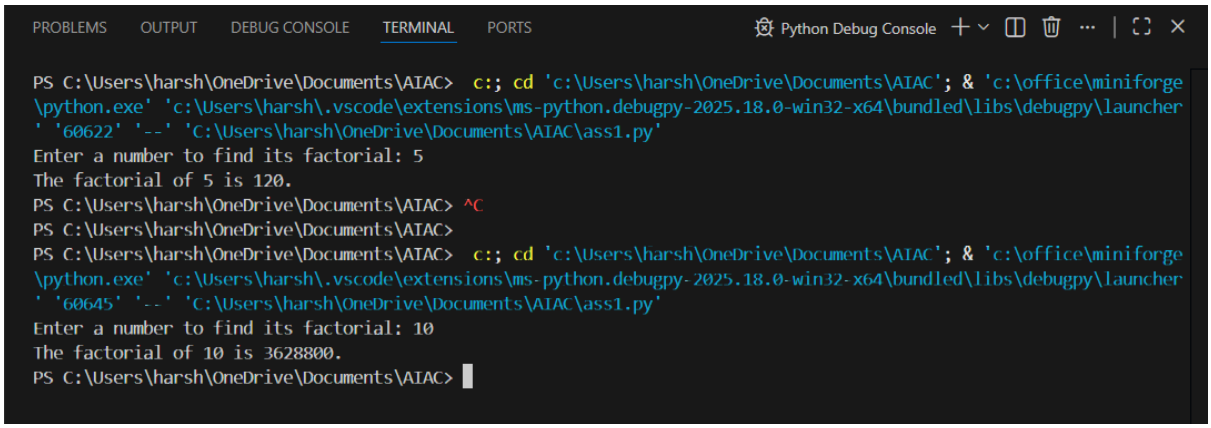
• Compare the generated code with a zero-shot solution.

• Examine improvements in clarity and correctness.

PROMPT



OUTPUT



ANALYSIS

- One-shot prompting produces better and safer code than zero-shot prompting.
- The example helps the AI write clearer logic and handle edge cases correctly.

Question 3: Few-Shot Prompting (Armstrong Number Check) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

• Input: 153 → Output: Armstrong Number

• Input: 370 → Output: Armstrong Number

• Input: 123 → Output: Not an Armstrong Number

Task:

• Analyze how multiple examples influence code structure and

accuracy.

• Test the function with boundary values and invalid inputs.

PROMPT

```python
#check whether a number is amstrong or not
def is_armstrong():
    num = int(input("Enter a number: "))
    original_num = num
    num_digits = len(str(num))
    sum_of_powers = 0

    while num > 0:
        digit = num % 10
        sum_of_powers += digit ** num_digits
        num //= 10

    if original_num == sum_of_powers:
        print(f"{original_num} is an Armstrong number.")
    else:
        print(f"{original_num} is not an Armstrong number.")
is_armstrong()
```

OUTPUT

```
PS C:\Users\harsh\OneDrive\Documents\AIAC>  & 'c:\office\miniforge\python.exe' 'c:\Users\harsh\.vscode\extens
ions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64266' '--' 'C:\Users\harsh\OneDri
ve\Documents\AIAC\ass1.py'
Enter a number: 151
151 is not an Armstrong number.
PS C:\Users\harsh\OneDrive\Documents\AIAC>
```

ANALYSIS

- The code structure is better and follows the examples.
- The code does not handle negative or invalid inputs properly.
- Accuracy is higher compared to zero-shot and one-shot prompting.

Question 4: Context-Managed Prompting (Optimized Number Classification) Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

• Ensure proper input validation.

• Optimize the logic for efficiency.

• Compare the output with earlier prompting strategies.

PROMPT

```python
# take a input frome user as a number and check whether the number is prime or composite or
def check_prime_composite():
    num = int(input("Enter a number: "))

    if num <= 1:
        print(f"{num} is neither prime nor composite.")
        return

    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            print(f"{num} is a composite number.")
            return

    print(f"{num} is a prime number.")
check_prime_composite()
```

OUTPUT

```
gpy\launcher' '53778' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 2
2 is a prime number.
PS C:\Users\harsh\OneDrive\Documents\AIAC> ^C
PS C:\Users\harsh\OneDrive\Documents\AIAC>
PS C:\Users\harsh\OneDrive\Documents\AIAC>  c:; cd 'c:\Users\harsh\OneDrive\Documents\AIAC'; & 'c:\office\min
iforge\python.exe' 'c:\Users\harsh\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
gpy\launcher' '53798' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 345
345 is a composite number.
PS C:\Users\harsh\OneDrive\Documents\AIAC> ^C
PS C:\Users\harsh\OneDrive\Documents\AIAC>
PS C:\Users\harsh\OneDrive\Documents\AIAC>  c:; cd 'c:\Users\harsh\OneDrive\Documents\AIAC'; & 'c:\office\min
iforge\python.exe' 'c:\Users\harsh\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
gpy\launcher' '56708' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 0
0 is neither prime nor composite.
PS C:\Users\harsh\OneDrive\Documents\AIAC>
```

ANALYSIS

- Input validation is properly handled.
- Compared to earlier prompting methods, this gives the best accuracy and performance.
- Clear instructions helped the AI write correct and efficient code.

Question 5: Zero-Shot Prompting (Perfect Number Check) Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

• Record the AI-generated code.

• Test the program with multiple inputs.

• Identify any missing conditions or inefficiencies in the logic.

PROMPT

```python
# take a input from user as a number and check whether the number is perfect number or not
def is_perfect_number():
    num = int(input("Enter a number: "))
    sum_of_divisors = 0

    for i in range(1, num):
        if num % i == 0:
            sum_of_divisors += i

    if sum_of_divisors == num:
        print(f"{num} is a perfect number.")
    else:
        print(f"{num} is not a perfect number.")
is_perfect_number()
```

OUTPUT

```
gpy\launcher' '49321' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 344
344 is not a perfect number.
PS C:\Users\harsh\OneDrive\Documents\AIAC> ^C
PS C:\Users\harsh\OneDrive\Documents\AIAC>
PS C:\Users\harsh\OneDrive\Documents\AIAC>  c:; cd 'c:\Users\harsh\OneDrive\Documents\AIAC'; & 'c:\office\min
iforge\python.exe' 'c:\Users\harsh\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
gpy\launcher' '49342' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 23
23 is not a perfect number.
PS C:\Users\harsh\OneDrive\Documents\AIAC>
```

ANALYSIS

The zero-shot prompt created a correct program for checking perfect numbers.
The program works well for normal positive numbers like 6 and 28.
It does not handle 0, negative numbers, or wrong inputs properly.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

• Input: 8 → Output: Even

• Input: 15 → Output: Odd

• Input: 0 → Output: Even

Task:

• Analyze how examples improve input handling and output

clarity.

• Test the program with negative numbers and non-integer inputs.

PROMPT

```python
# take a input from user as a number and check whether the number is even or odd and it shou
def check_even_odd():
    try:
        num = int(input("Enter a number: "))

        if num < 0:
            print("Please enter a non-negative integer.")
            return

        if num % 2 == 0:
            print(f"{num} is an even number.")
        else:
            print(f"{num} is an odd number.")
    except ValueError:
        print("Invalid input. Please enter an integer.")
check_even_odd()
```

OUTPUT

```
Enter a number: 0
0 is an even number.
PS C:\Users\harsh\OneDrive\Documents\AIAC> ^C
PS C:\Users\harsh\OneDrive\Documents\AIAC>
PS C:\Users\harsh\OneDrive\Documents\AIAC>  c:; cd 'c:\Users\harsh\OneDrive\Documents\AIAC'; & 'c:\office\min
iforge\python.exe' 'c:\Users\harsh\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
gpy\launcher' '53143' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: -7
Please enter a non-negative integer.
PS C:\Users\harsh\OneDrive\Documents\AIAC> ^C
PS C:\Users\harsh\OneDrive\Documents\AIAC>
PS C:\Users\harsh\OneDrive\Documents\AIAC>  c:; cd 'c:\Users\harsh\OneDrive\Documents\AIAC'; & 'c:\office\min
iforge\python.exe' 'c:\Users\harsh\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
gpy\launcher' '53163' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 7
7 is an odd number.
PS C:\Users\harsh\OneDrive\Documents\AIAC> ^C
PS C:\Users\harsh\OneDrive\Documents\AIAC>
PS C:\Users\harsh\OneDrive\Documents\AIAC>  c:; cd 'c:\Users\harsh\OneDrive\Documents\AIAC'; & 'c:\office\min
iforge\python.exe' 'c:\Users\harsh\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
gpy\launcher' '53182' '--' 'C:\Users\harsh\OneDrive\Documents\AIAC\ass1.py'
Enter a number: 67.8
Invalid input. Please enter an integer.
PS C:\Users\harsh\OneDrive\Documents\AIAC>
```

ANALYSIS

The given examples helped the AI understand what even and odd mean.
The program gives clear output like “Even” or “Odd”.
It correctly works for negative numbers also.
It safely rejects non-integer inputs like decimals or strings.